



리눅스 기본 명령어 학습

학습 목표

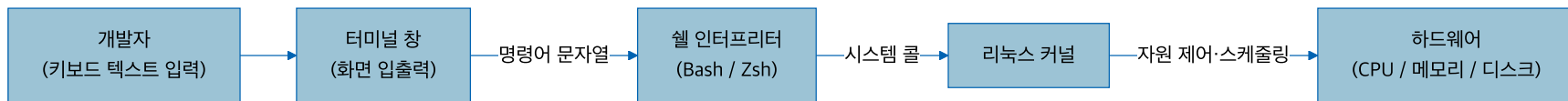
- 컨테이너 내부 쉘에서 패키지 관리자로 필요한 도구를 설치합니다
- 현재 위치를 확인하고 작업 디렉터리를 만들어 이동합니다
- 환경변수와 리다이렉션으로 설정 파일을 작성하고 내용을 검색합니다
- 파일 권한 오류를 재현하고 `chmod`로 실행 권한을 부여해 해결합니다

클라우드 가상 서버, 도커 컨테이너, CI/CD 러너 환경에는 그래픽 인터페이스가 없습니다. 텍스트 기반 명령어 인터페이스로 파일을 만들고, 내려받고, 권한을 조정해 스크립트를 실행하는 기본 조작을 숙달하는 것이 본 차시의 목표입니다.

핵심 개념 및 구조

명령어가 전달되는 경로

터미널에 입력한 문자열은 곧바로 하드웨어를 제어하지 않습니다. 셸이 문자열을 해석해 커널에 시스템 콜로 전달하고, 커널이 하드웨어 자원을 제어합니다.



터미널

키보드 입력을 받아 화면에 출력하는
입출력 창구

셸

입력 문자열을 해석해 커널이 처리할
시스템 콜로 변환

커널

CPU·메모리·디스크 자원을 실제로
제어하고 스케줄링

CLI와 GUI의 특성 비교

비교 항목	GUI (Graphical User Interface)	CLI (Command Line Interface)
인터페이스	마우스 클릭, 그래픽 윈도우, 아이콘	키보드 텍스트 명령어, 터미널 프롬프트
자원 소모	그래픽 렌더링으로 CPU·메모리·대역폭 소모 큼	텍스트 스트림만 송수신하여 소모 극소
무인 자동화	화면 조작이 필요해 스크립트화 난해	셸 스크립트로 무인 자동화 및 CI/CD 연동
서버·클라우드	일반적인 운영 서버에는 미설치	모든 리눅스 서버·컨테이너의 표준 환경

원격 서버는 네트워크 너머에 있고 화면 장치가 없습니다. 텍스트만 오가는 CLI는 좁은 대역폭에서도 동작하며, 같은 명령어를 파일로 저장해 그대로 재실행할 수 있습니다.

운영체제 계보와 터미널 환경

운영체제는 크게 유닉스 계열과 윈도우 계열로 갈리며, 이 구분이 실습 명령어의 통용 범위를 결정합니다.

운영체제	계보	기본 셸	리눅스 명령어 사용
Unix	1970년대 원형 (AT&T)	<code>sh</code> 계열	표준의 원형
Linux	유닉스 호환 커널 재구현	<code>bash</code>	서버·컨테이너 표준
macOS	유닉스 계열 (BSD 기반)	<code>zsh</code>	대부분 동일하게 사용
Windows	독자 계열 (NT 커널)	<code>PowerShell</code>	<code>WSL2</code> · <code>Git Bash</code> 필요

컨테이너 내부에서 실습하면 호스트가 macOS든 Windows든 동일한 리눅스 환경이 보장됩니다.

리눅스 디렉터리 구조와 경로 표기

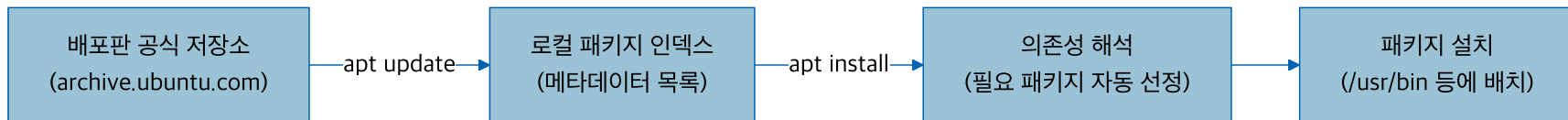
리눅스의 모든 경로는 최상위 루트(`/`)에서 시작하는 하나의 트리로 이어집니다. 드라이브 문자(`C:`)는 존재하지 않습니다.

경로	용도	실습에서의 의미
<code>/</code>	파일 시스템 최상위 루트	모든 절대 경로의 시작점
<code>/etc</code>	시스템 전역 설정 파일	서비스 설정이 놓이는 표준 위치
<code>/var</code>	로그·캐시 등 변동 데이터	<code>/var/log</code> 에서 운영 로그 확인
<code>/root</code>	root 계정 홈 디렉터리	컨테이너 접속 직후의 <code>\$HOME</code>
<code>/practice</code>	실습 전용 작업 디렉터리	<code>mkdir</code> 로 직접 생성해 사용

절대 경로는 `/`로 시작해 어디서든 같은 대상을 가리키고, **상대 경로**는 현재 디렉터리 기준으로 해석됩니다.

패키지 관리자의 동작 구조

컨테이너 기본 이미지에는 최소한의 도구만 들어 있습니다. 필요한 프로그램은 배포판 공식 저장소에서 패키지 관리자로 설치합니다.

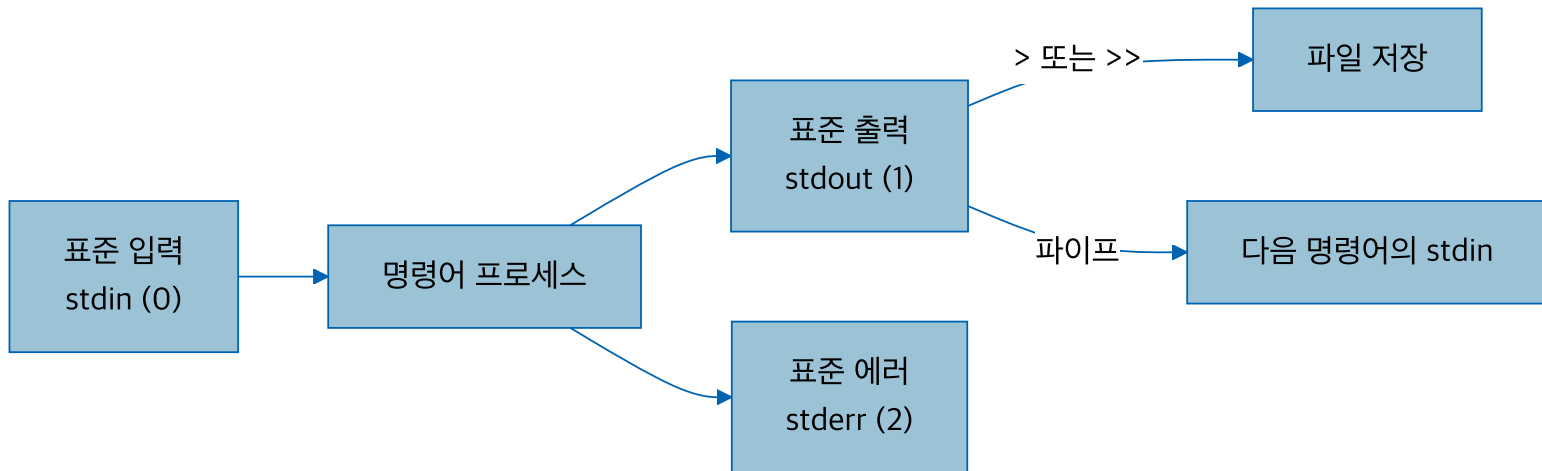


실습에서 사용하는 명령	수행하는 일
<code>apt update</code>	공식 저장소의 패키지 인덱스를 최신 상태로 동기화
<code>apt install -y curl</code>	의존성을 해석해 <code>curl</code> 패키지를 설치 (<code>-y</code> 는 확인 자동 승인)

인덱스를 갱신하지 않고 설치하면 `Unable to locate package` 오류가 발생합니다. 실습 환경은 Ubuntu이므로 `apt`를 사용합니다.

표준 입출력과 스트림

리눅스의 모든 명령어는 입력 하나와 출력 두 개의 통로를 갖습니다. 이 통로를 파일로 돌리거나 다른 명령어에 연결하는 것이 리다이렉션과 파이프입니다.



출력이 화면으로 나가는 것은 기본값일 뿐이며, 목적지는 파일이나 다른 명령어로 바꿀 수 있습니다.

리다이렉션과 파이프 연산자

기호	기능	동작 방식
>	덮어쓰기 리다이렉션	파일이 없으면 생성, 있으면 기존 내용을 모두 지우고 새로 씀
>>	이어쓰기 리다이렉션	파일이 없으면 생성, 있으면 기존 내용 끝에 새 줄 추가
	파이프	앞 명령어의 표준 출력을 뒤 명령어의 표준 입력으로 연결

>는 확인 절차 없이 기존 내용을 즉시 삭제합니다. 로그나 설정 파일에 내용을 덧붙일 때는 반드시 >>를 사용합니다.

환경변수의 역할

환경변수는 셸 세션과 그 하위 프로세스가 공유하는 설정값입니다. 코드를 수정하지 않고 실행 환경만 바꾸는 표준 수단입니다.

구분	명령	적용 범위
값 조회	<code>echo \$HOME</code> / <code>printenv APP_ENV</code>	현재 셸에 등록된 값 출력
세션 변수	<code>APP_ENV=development</code>	현재 셸에서만 유효, 하위 프로세스에 미전달
환경변수 등록	<code>export APP_ENV=development</code>	현재 셸과 이후 실행되는 모든 하위 프로세스에 전달

자격증명과 접속 정보를 코드에 직접 적지 않고 환경변수로 주입하는 방식은 12-Factor App의 기본 원칙이며, 추후 컨테이너 설정 주입 실습으로 그대로 이어집니다.

파일 권한 체계

리눅스는 파일마다 소유자·그룹·기타 사용자 세 주체에 대해 읽기·쓰기·실행 권한을 따로 관리합니다.

권한 비트	기호	8진수 값	파일에 적용될 때	디렉터리에 적용될 때
Read	<code>r</code>	<code>4</code>	내용 열람	목록 조회 (<code>ls</code>)
Write	<code>w</code>	<code>2</code>	내용 수정	내부 파일 생성·삭제
Execute	<code>x</code>	<code>1</code>	스크립트·바이너리 실행	디렉터리 진입 (<code>cd</code>)

각 주체의 권한 값을 더해 한 자리 숫자로 표기합니다. 소유자 `rwX` 는 $4+2+1=7$, 그룹 `r-x` 는 $4+0+1=5$ 입니다.

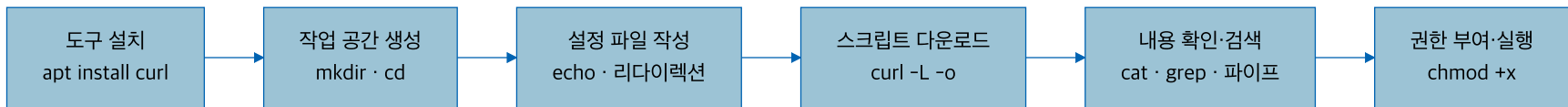
권한 표기 읽는 법

`ls -l`이 출력하는 열 칸짜리 문자열은 파일 종류 한 칸과 세 주체의 권한 세 묶음으로 구성됩니다.

표기	파일 종류	소유자	그룹	기타	8진수	용도
<code>-rwxr-xr-x</code>	일반 파일	<code>rwX</code>	<code>r-x</code>	<code>r-x</code>	<code>755</code>	실행 스크립트 표준
<code>-rw-r--r--</code>	일반 파일	<code>rw-</code>	<code>r--</code>	<code>r--</code>	<code>644</code>	일반 설정 파일 표준
<code>drwxr-xr-x</code>	디렉터리	<code>rwX</code>	<code>r-x</code>	<code>r-x</code>	<code>755</code>	일반 디렉터리 표준

웹에서 내려받은 파일에는 실행 비트가 없어 `644` 상태로 생성됩니다. 그대로 실행하면 `Permission denied`가 발생하며, 이는 파일이 손상된 것이 아니라 권한이 없다는 의미입니다.

실습 전체 흐름: 도구 설치부터 권한 부여까지



환경 준비

`apt`, `pwd`, `mkdir`, `cd`

필요한 도구를 설치하고 작업 디렉터리를 만든다

파일 생성과 조회

`echo`, `>`, `cat`, `grep`

설정 파일을 작성하고 필요한 줄만 검색한다

권한과 정리

`chmod`, `cp`, `mv`, `rm`

실행 권한을 부여하고 실습 자원을 정리한다

실습 단계

실습 환경 확인

실습은 앞서 실행한 Ubuntu 컨테이너 내부 셸에서 진행합니다. 프롬프트가 `root@<container-id>:/#` 형태이면 컨테이너 내부입니다.

```
# 셸이 종료된 경우 호스트 터미널에서 컨테이너를 다시 시작하고 접속한다
docker start -ai linux-practice

# 컨테이너 내부인지 프롬프트와 커널 정보로 확인한다
whoami
uname -a
```

호스트가 macOS나 Windows여도 컨테이너 내부는 동일한 리눅스 환경입니다. 명령어 동작과 파일 권한 체계가 모든 학습자에게 같게 적용됩니다.

실습 로드맵

Step	주제	이 단계에서 직접 확인하는 것
1	패키지 관리자로 <code>curl</code> 설치	저장소 인덱스 갱신과 의존성 자동 설치 절차
2	위치 확인과 작업 디렉터리 생성	절대 경로 출력과 디렉터리 생성·이동
3	환경변수와 설정 파일 작성	<code>export</code> 등록과 리다이렉션 <code>></code> · <code>>></code> 의 차이
4	<code>curl</code> 로 배포 스크립트 다운로드	리다이렉트 추적과 파일명 지정 저장
5	파일 내용 확인과 문자열 검색	<code>cat</code> · <code>head</code> · <code>tail</code> · <code>grep</code> 과 파이프 연결
6	권한 오류 재현과 <code>chmod</code> 해결	<code>Permission denied</code> 원인 규명과 실행 비트 부여
7	복사·이동·삭제와 컨테이너 정리	파일 조작 명령과 컨테이너 인스턴스 제거

명령어 한눈에 보기: 환경 준비와 파일 작성

도구 설치와 작업 공간 Step 1~2

```
# 1. 저장소 인덱스 갱신 후 curl 설치
apt update
apt install -y curl
curl --version

# 2. 위치 확인과 작업 디렉터리 생성
pwd
ls
ls -la
mkdir /practice
cd /practice
pwd
```

환경변수·설정 파일과 다운로드 Step 3~4

```
# 3. 환경변수 등록과 설정 파일 작성
echo "Hello Linux"
echo $HOME
export APP_ENV=development
printenv APP_ENV
echo "APP_ENV=development" > app.env
echo "PORT=8080" >> app.env
cat app.env

# 4. 배포 스크립트 다운로드
curl -L -o deploy.sh <배포 스크립트 URL>
ls -l
```

명령어 한눈에 보기: 검색·권한·정리

내용 확인과 검색 Step 5

```
# 파일 전체·상단·하단 확인
cat app.env
cat deploy.sh
head -n 3 deploy.sh
tail -n 3 deploy.sh

# 문자열 검색과 파이프 연결
grep "APP_" app.env
grep -n "echo" deploy.sh
cat deploy.sh | grep -i "server"
```

권한 부여와 정리 Step 6~7

```
# 권한 오류 재현 후 실행 비트 부여
ls -l deploy.sh
./deploy.sh
chmod +x deploy.sh
./deploy.sh
chmod 644 app.env

# 복사·이동·삭제 후 컨테이너 정리
cp deploy.sh deploy-backup.sh
mv deploy-backup.sh archived.sh
rm archived.sh
exit # 컨테이너 셸 종료
docker rm linux-practice # 호스트 터미널에서 실행
```

Step별 이해 포인트

Step	반드시 이해하고 넘어갈 핵심 포인트
1	<code>apt update</code> 는 패키지를 설치하지 않고 인덱스만 갱신합니다. 설치하는 <code>apt install</code> 이 수행합니다.
2	<code>ls</code> 는 일반 파일만, <code>ls -la</code> 는 숨김 파일과 권한·소유자·크기까지 출력합니다.
3	<code>></code> 는 기존 내용을 지우고 덮어쓰며, <code>>></code> 는 끝에 덧붙입니다. 순서를 바꾸면 앞 줄이 사라집니다.
4	<code>-L</code> 은 리다이렉트를 추적하고 <code>-o</code> 는 저장 파일명을 지정합니다. <code>-o</code> 가 없으면 내용이 화면으로 출력됩니다.
5	<code>grep</code> 은 파일을 직접 받을 수도 있고, 파이프로 앞 명령어의 출력을 받을 수도 있습니다.
6	실행 실패의 원인은 파일 손상이 아니라 실행 비트(<code>x</code>) 부재입니다.
7	<code>rm</code> 은 휴지통 없이 즉시 삭제하며, <code>docker rm</code> 은 컨테이너 인스턴스 자체를 제거합니다.

권한 오류 재현과 해결

내려받은 스크립트가 실행되지 않는 상황을 재현하고 권한 상태 변화를 확인합니다. 현재 디렉터리는 `PATH`에 포함되지 않으므로 실행 시 `./`를 붙입니다.

```
ls -l deploy.sh      # -rw-r--r-- : 실행 비트 없음
./deploy.sh          # bash: ./deploy.sh: Permission denied (Exit Code 126)
chmod +x deploy.sh   # 실행 비트 추가
ls -l deploy.sh      # -rwxr-xr-x : 실행 가능 상태
./deploy.sh          # 정상 실행
```

명령	표기법	결과 권한
<code>chmod +x deploy.sh</code>	기호 모드	기존 권한에 실행 비트만 추가
<code>chmod 755 deploy.sh</code>	숫자 모드	소유자 <code>rwx</code> , 그룹 <code>r-x</code> , 기타 <code>r-x</code>
<code>chmod 644 app.env</code>	숫자 모드	소유자 <code>rw-</code> , 그룹 <code>r--</code> , 기타 <code>r--</code>

한 장 요약

영역	핵심 요약	비고
명령 전달 구조	터미널 → 셸 해석 → 커널 시스템 콜 → 하드웨어 제어	CLI는 서버·CI/CD의 표준 인터페이스
패키지 관리	인덱스 갱신 후 의존성 해석 설치	<code>apt update</code> → <code>apt install -y</code>
입출력 제어	표준 출력의 목적지를 파일·다음 명령어로 전환	<code>></code> 덮어쓰기, <code>>></code> 이어쓰기, 파이프 연결
파일 권한	소유자·그룹·기타별 <code>r</code> · <code>w</code> · <code>x</code> 비트를 8진수로 표기	<code>755</code> 실행 스크립트, <code>644</code> 설정 파일

파일을 만들고, 내려받고, 검색하고, 권한을 조정해 실행하는 절차는 이후 모든 컨테이너·배포 실습의 기본 조작입니다.

Appendix

배포판별 패키지 관리자

배포판 계열	패키지 관리자	인덱스 갱신	설치 명령
Ubuntu / Debian	apt	apt update	apt install -y curl
Red Hat / Rocky	dnf	dnf makecache	dnf install -y curl
Alpine	apk	apk update	apk add curl

명령어 이름과 옵션은 다르지만 공식 저장소의 인덱스를 갱신하고 의존성을 해석해 설치한다는 절차는 동일합니다. 컨테이너 기본 이미지가 바뀌면 이 표를 기준으로 설치 명령을 교체합니다.

선택 명령과 확장 옵션

명령	기능	사용 상황
<code>curl -s URL > 파일명</code>	진행 표시 없이 본문만 저장	스크립트 내부에서 출력을 깔끔하게 유지할 때
<code>curl -I URL</code>	HTTP 응답 헤더만 조회	상태 코드와 리다이렉트 경로를 빠르게 점검할 때
<code>chown 사용자:그룹 파일명</code>	파일 소유자와 그룹 변경	권한 비트가 아니라 소유 주체를 바꿔야 할 때
<code>find /경로 -name "*.sh"</code>	조건에 맞는 파일 탐색	위치를 모르는 파일을 경로 전체에서 찾을 때
<code>tail -f 로그파일</code>	파일 끝을 실시간 추적 출력	운영 중인 서비스의 로그를 즉시 확인할 때

Docker 기반 Ubuntu 쉘은 Windows와 macOS의 명령 체계 및 파일 권한 차이를 제거해, 모든 학습자에게 동일한 실습 환경을 제공합니다.

핵심 기술 면접 질문

GUI와 CLI의 차이

GUI와 CLI의 차이는 무엇이며, 서버 및 CI/CD 환경에서 CLI를 선호하는 이유는 무엇인가요?

커널과 셸의 역할과 차이

운영체제에서 커널과 셸의 역할과 차이는 무엇인가요?

`docker run -it` 옵션의 의미

`docker run -it` 옵션의 의미와 컨테이너 내부 셸 진입 원리는 무엇인가요?

`ls -l`과 `ls -a` 옵션의 차이

`ls -l`과 `ls -a` 옵션은 각각 어떤 정보를 보여주나요?

핵심 기술 면접 질문

리다이렉션 `>`와 `>>`의 차이

리다이렉션 연산자 `>`와 `>>`의 차이는 무엇인가요?

`curl` 주요 옵션의 용도

`curl`에서 `-L`, `-O`, `-s`, `-I` 옵션은 각각 언제 사용하나요?

파일 권한 표기와 8진수 계산

리눅스 파일 권한 표기(예: `-rwxr-xr--`)와 8진수 표기(예: `755`)는 어떻게 계산하나요?

`Permission denied` 원인과 해결

스크립트 실행 시 `Permission denied` 오류가 발생하는 원인과 `chmod`를 통한 해결 방법은 무엇인가요?

핵심 기술 면접 질문

`chmod`와 `chown`의 차이

`chmod`와 `chown`의 차이는 무엇인가요?

로그 분석 핵심 명령어의 활용

`cat`, `head`, `tail`, `grep`은 실무 로그 분석에서 각각 어떤 상황에 쓰이나요?

모범 답변

GUI와 CLI의 차이

GUI는 시각적이고 직관적이지만 네트워크 대역폭과 시스템 자원을 많이 소모하며 무인 자동화가 어렵습니다. CLI는 텍스트 기반으로 자원 소모가 거의 없고, 셸 스크립트나 CI/CD 파이프라인을 통한 원격 제어와 무인 자동화에 필수적입니다.

커널과 셸의 차이

커널은 CPU, 메모리, 디스크 등 하드웨어 자원을 직접 제어하고 스케줄링하는 운영체제의 핵심 엔진입니다. 셸은 사용자가 입력한 텍스트 명령어를 해석해 커널이 실행할 수 있는 시스템 콜로 전달하는 인터페이스 프로그램입니다.

`docker run -it` 옵션의 의미

`-i` (interactive)는 표준 입력을 열어두어 명령을 전달할 수 있게 하고, `-t` (tty)는 가상 터미널을 할당해 프롬프트가 표시되게 합니다. 두 옵션을 함께 사용해 컨테이너 내부 Bash 셸에 직접 접속합니다.

모범 답변

ls -l 과 ls -a 옵션의 차이

`-l` (long)은 파일 크기, 권한, 소유자, 최종 수정일 등 상세 속성을 열 형태로 출력합니다. `-a` (all)는 마침표(`.`)로 시작하는 숨김 파일과 디렉터리를 포함하여 모든 항목을 빠짐없이 표시합니다.

리다이렉션 `>`와 `>>`의 차이

`>`는 대상 파일이 없으면 새로 만들고, 이미 존재하면 기존 내용을 전부 지운 뒤 새로 덮어씁니다(Overwrite). `>>`는 파일이 없으면 새로 만들고, 이미 존재하면 기존 내용의 끝에 새 줄을 덧붙입니다(Append).

curl 주요 옵션의 용도

`-L`은 리다이렉트(301/302) 발생 시 최종 목적지까지 추적합니다. `-o [파일명]`은 내려받은 데이터를 지정한 이름으로 저장합니다. `-s`는 진행률 표시를 숨겨 응답 데이터만 출력합니다. `-I`는 본문 없이 HTTP 응답 헤더만 조회합니다.

모범 답변

파일 권한 표기와 8진수 계산

권한은 읽기(**r**: 4), 쓰기(**w**: 2), 실행(**x**: 1)의 합으로 계산하며 소유자·그룹·기타 순으로 세 자리로 표기합니다. 소유자가 **rwX** (7), 그룹이 **r-x** (5), 기타가 **r-x** (5)이면 **755**가 됩니다.

Permission denied 원인과 해결

웹이나 외부에서 내려받은 파일은 보안상 실행 비트(**x**)가 제거된 일반 텍스트 상태(**644**, **-rw-r--r--**)로 생성됩니다. 이를 실행 파일로 구동하려면 **chmod +x [파일명]** 또는 **chmod 755 [파일명]**으로 실행 권한을 부여해야 합니다.

모범 답변

`chmod`와 `chown`의 차이

`chmod` (Change Mode)는 파일의 읽기·쓰기·실행 접근 권한 비트를 변경하고, `chown` (Change Owner)은 파일의 소유자 (User)나 소유 그룹(Group) 자체를 변경합니다.

로그 분석 핵심 명령어의 활용

`cat`은 짧은 설정 파일 전체를 열람할 때, `head`는 파일 상단의 헤더와 선언부를 확인할 때, `tail`은 로그 파일의 최신 기록을 확인할 때, `grep`은 대용량 로그에서 특정 키워드나 에러 문자열만 필터링할 때 사용합니다.

깊이 알아보기

셸 공식 매뉴얼 GNU Bash Manual
리다이렉션·환경변수·확장 규칙 원문

<https://www.gnu.org/software/bash/manual/bash.html>

기본 명령어 모음 GNU Coreutils
ls · cat · cp · mv · rm 공식 사양

<https://www.gnu.org/software/coreutils/manual/coreutils.html>

디렉터리 구조 표준 FHS 3.0
리눅스 파일 시스템 계층 표준 원문

https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

권한 변경 명령 chmod(1)
기호 모드·숫자 모드 공식 매뉴얼

<https://man7.org/linux/man-pages/man1/chmod.1.html>

깊이 알아보기

문자열 검색 명령 `grep(1)`

정규표현식 검색 옵션 공식 매뉴얼

<https://man7.org/linux/man-pages/man1/grep.1.html>

네트워크 전송 도구 `curl` Manual

전체 옵션 목록과 사용 예시 원문

<https://curl.se/docs/manpage.html>



- EOF -